Swarm Search

Development Report

Implementation details, milestones, current development stage, and planned next steps

---

Executive Summary

Swarm Search is a multi-agent reinforcement-learning environment simulating a team of drones searching a grid-based area, inspired by search-and-rescue-style coverage missions. The project currently combines a PettingZoo-compliant simulation core, real-world obstacle data pulled from OpenStreetMap, several swappable movement strategies, and two live visualizations -- a desktop matplotlib viewer and a browser-based live map.

This report documents what has actually been built and verified so far, distinguishes clearly between mechanisms that look similar but serve different purposes (most importantly: generating random obstacles for training variety, versus recognizing obstacles from unknown terrain, which is not yet implemented), and lays out the specific next steps planned.

Current stage, in one sentence

The simulation environment, real-world data integration, and two non-learned coordination strategies (potential fields and BFS pathfinding) are built and tested; no trained neural-network policy exists yet, and drones do not yet model realistic communication constraints -- both are the next major milestones (see 'Planned Future Work').

System Architecture

The codebase is organized in layers, with the core simulation kept independent of any specific decision-making strategy or visualization -- swapping in a trained policy later should not require changing the environment itself.

File map

```
swarm-env/swarm.py          Core RL environment (SwarmCoverageEnv)
swarm-env/osm_obstacles.py  Real-world obstacle extraction from OpenStreetMap
swarm-env/basemap.py        Real basemap tile images for the desktop viewer
movement_policy.py          Decision-making strategies (random / potential
                            field / BFS pathfinding)
visualize_swarm.py          Desktop live viewer (matplotlib)
server.py                   Live web backend (FastAPI + WebSocket)
web/index.html, web/app.js  Live web frontend (Leaflet map)
```

Design principle: environment vs. decision-maker are separate

SwarmCoverageEnv only knows how to apply an action and compute the resulting state/reward -- it never decides what a drone should do. Every visualization and the web server call out to a separate policy object (random sampling, PotentialFieldPolicy, or BFSCoveragePolicy) for that decision. This means a future trained policy plugs in at exactly the same point, with no changes required to the environment, obstacle system, or visualizations.

Implementation: Core Environment

SwarmCoverageEnv (swarm-env/swarm.py) implements PettingZoo's ParallelEnv interface, so it is directly compatible with standard multi-agent RL tooling (TorchRL's PettingZoo wrapper, in particular).

Grid and state

- A grid_size x grid_size grid where each cell is one of: obstacle, covered (visited by any drone this episode), or uncovered free space.
- Each drone occupies one cell and picks one of 5 discrete actions per step: up, down, left, right, or stay.
- reset() places obstacles, gives each drone a random free starting cell, and marks those starting cells as covered.

Observation: local only, by design

Each drone's observation is a (2*obs_window+1) x (2*obs_window+1) patch centered on itself (default 7x7), where each cell reads -1 for obstacle/off-grid, 1 for covered, 0 for uncovered -- plus its own normalized position. It cannot see the whole grid or where teammates currently are. This mirrors a real drone only knowing what its own onboard sensors can detect.

Reward: shared, coverage-based

```
reward = (number of newly covered cells this step) - step_penalty
```

This reward is identical for every drone regardless of which one actually reached new ground -- a design choice meant to push a future trained policy toward spreading out, since two drones covering the same cell contribute zero combined new coverage.

Battery: real, tracked per-drone state

Each drone starts an episode at 100% battery. Moving drains more per step than staying idle (1.0% vs 0.2% by default), floored at 0%. Battery is returned in the info dict every step and is exactly the kind of compact status a real drone would broadcast over radio (see 'How Drones Communicate Today').

Obstacles: Two Distinct Mechanisms

This project currently has two entirely different mechanisms that both result in 'obstacles' on the grid, and they serve different purposes. Conflating them would misrepresent what's actually implemented, so they're kept explicitly separate here.

Mechanism 1: random obstacle generation (a training-data mechanic)

When no real-world bounding box is given, reset() scatters n_obstacles randomly across the grid, differently every single episode. This exists purely to create map variety during training -- if a policy only ever trained on one fixed layout, it would risk memorizing that exact map instead of learning a general search strategy. This is standard practice in RL environment design and has nothing to do with perception or sensing.

Mechanism 2: real-world obstacle placement from OpenStreetMap

When a real lon/lat bounding box is given, osm_obstacles.py downloads actual building/water/forest geometry for that area from OpenStreetMap and rasterizes it onto the grid (with a minimum-area-overlap threshold, so a cell only counts as blocked if a meaningful fraction of it is actually covered by a real feature -- an early version used a simple touch-test and it drastically overcounted obstacles at city scale). This layout is fixed across episodes, since it represents one specific real place.

What is NOT yet implemented: obstacle recognition

Both mechanisms above hand the drone ground-truth obstacle information directly -- the local observation patch simply looks up which nearby cells are marked as obstacles in the environment's internal, perfect-knowledge grid. There is no perception step: no simulated sensor data, no noise, no classification of 'is this an obstacle' from indirect signals, and no ability to generalize to terrain the system has never encountered before in any sensor-realistic sense. This is a lookup, not recognition. Building an actual recognition process -- inferring obstacles on previously unseen terrain from imagery or simulated sensor input rather than from a hidden ground-truth grid -- is planned future work (see 'Planned Future Work' section).

Movement Strategies

Three movement strategies exist today, all in movement_policy.py, each usable as a drop-in replacement for the others without touching the environment.

1. Random (baseline)

Every drone samples a uniformly random action every step, ignoring its observation entirely. Used only as a lower-bound baseline for comparison.

2. Potential fields (hand-coded, force-based)

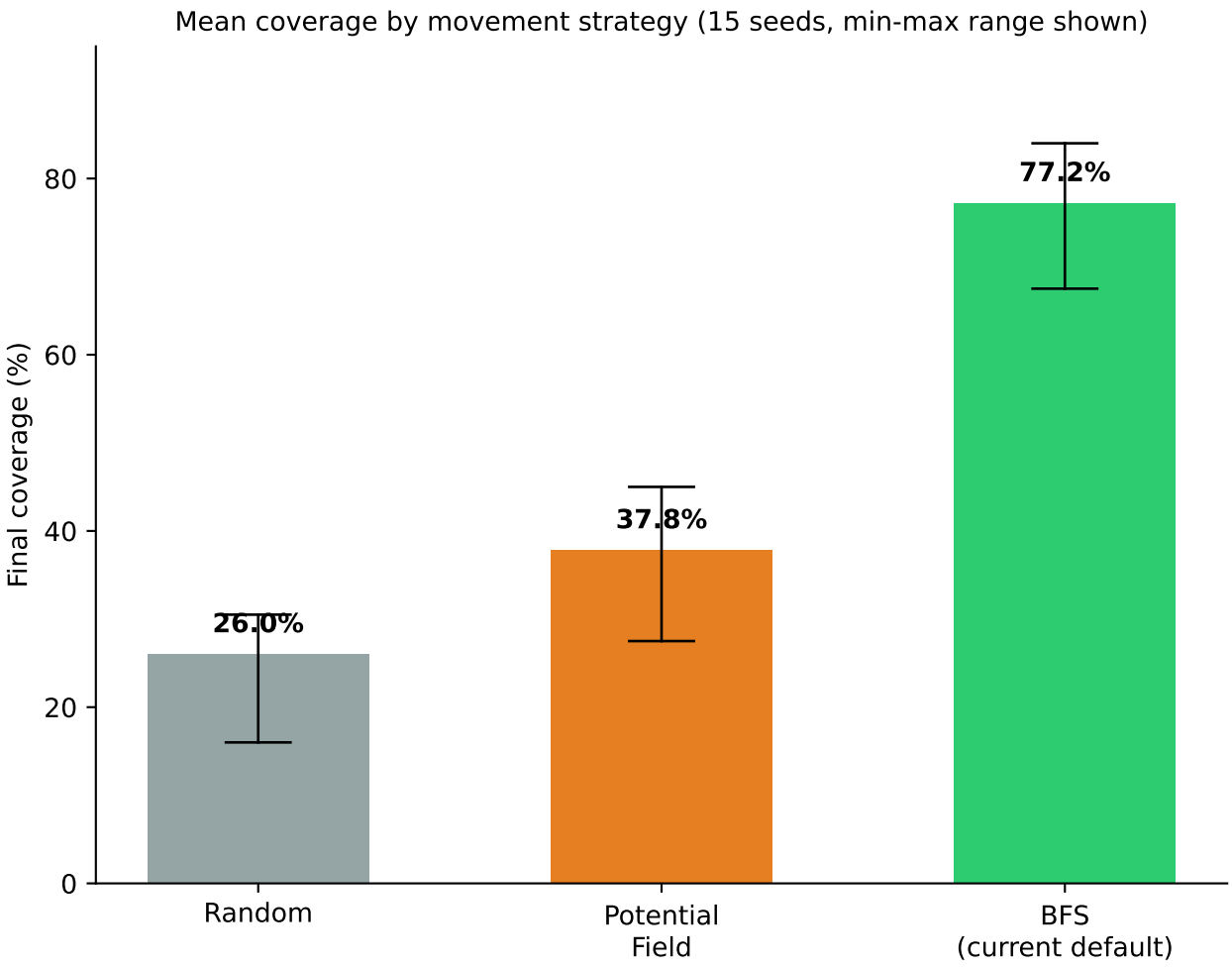
Each drone sums three force vectors -- repulsion from visible obstacles, repulsion from nearby teammates, attraction toward visible uncovered ground -- and moves in the resulting direction. This is a standard, decades-old robotics technique, not something novel to this project. It has a well-documented weakness: forces can reach a point where they cancel out, causing a drone to oscillate between two cells indefinitely. Several targeted fixes were applied (anti-backtracking memory, random jitter, handling drones sharing a cell, filtering out illegal moves before committing to them), but new local minima kept resurfacing because the failure mode is structural to the approach, not a specific bug.

3. BFS coverage (graph search, current default)

Each drone runs a real breadth-first search over free grid cells to find the true shortest-path distance to every reachable cell, then moves one step along the shortest path toward its nearest reachable uncovered cell. Drones claim disjoint target cells each step so the swarm naturally divides territory without needing a repulsion force. Because 'closer' is a strict integer hop count rather than a force that can cancel out, this cannot oscillate the way potential fields can.

Measured comparison

Averaged over 15 randomized 15x15 grids (3 drones, 25 obstacles, 60-step episodes):



BFS pathfinding roughly doubles coverage versus potential fields, and nearly triples it versus random action selection.

Visualization

Two independent ways to watch a simulation run live have been built. Neither depends on the other, and both read the same SwarmCoverageEnv/policy objects.

Desktop viewer (visualize_swarm.py)

- matplotlib-based, updates live via plt.pause() each step.
- Drones render as a small rotating triangular icon that turns to face whichever direction it actually just moved.
- Optionally draws a real basemap image underneath the grid (via the contextily library, pulling free OpenStreetMap/Esri tiles -- no API key needed) when the environment was built against a real bounding box.
- Displays each drone's live battery percentage as a small label, turning red under 30%.

Live web map (server.py + web/)

A FastAPI backend runs SwarmCoverageEnv continuously in a background loop (auto-resetting when an episode ends) and broadcasts each step's state as JSON over a WebSocket: every drone's real latitude/longitude (converted from its grid cell via SwarmCoverageEnv.cell_to_latlon), heading, and battery, plus which cells were newly covered.

The frontend is a Leaflet map (free OpenStreetMap tiles, no API key) that draws real building obstacles as dark overlays, shades newly covered cells green as they're reported, and moves a rotating triangle marker per drone. A sidebar shows live step count, overall coverage percentage, and a battery bar per drone.

This was chosen over the Google Maps JavaScript API specifically to avoid requiring billing/API key setup for a hackathon-stage project -- the visual result (real streets, real buildings) is equivalent for this purpose.

How Drones Communicate Today

This section is deliberately explicit about what 'communication' currently means in this codebase, because it is meaningfully different from how real drone swarms communicate, and that gap is one of the planned next steps.

What real drone swarms have to do

Real drones are separate physical devices with no shared memory. They must transmit state to each other over radio -- via mesh networking (messages hop drone to drone), direct point-to-point RF, or satellite/cellular fallback at longer range -- and every one of those channels has limited bandwidth, limited range, and can drop or delay messages. A real drone only knows what it has actually received, never the true global state of the swarm.

What this simulator actually does today

There is no message-passing model at all. SwarmCoverageEnv is a single Python object; every drone's position, the shared coverage map, and every drone's battery level are plain attributes on that one object, readable instantly by anything with a reference to it, with zero latency and zero data loss. When the reward function says 'shared,' it means every drone's reward is computed from that same global, always-current state -- not that drones exchanged any message to arrive at shared knowledge.

The BFS coverage policy's territory-claiming (each drone picks a distinct target cell so the swarm doesn't duplicate effort) and the potential-field policy's teammate repulsion both work by directly reading every drone's exact position off that same global object. This is explicitly flagged as a simplification in the code: it is a reasonable stand-in for 'drones broadcast GPS position, which is cheap and small,' but it is not an actual model of a radio channel, and it currently has no range limit, no packet loss, and no delay of any kind.

Net effect

Coordination today comes from every drone effectively having omniscient, instantaneous knowledge of the swarm's shared state, not from any simulated act of communicating. Modeling the real constraints -- range, bandwidth, dropped messages, per-drone partial knowledge -- is planned future work (see next section).

Development Milestones

Milestone 1 -- Core environment

Built SwarmCoverageEnv as a PettingZoo ParallelEnv: grid, local observations, shared coverage reward, random obstacle generation, discrete 5-action movement. Verified against PettingZoo's official API compliance test.

Milestone 2 -- Real-world grounding

Added osm_obstacles.py to pull real building/water/forest geometry from OpenStreetMap and rasterize it onto the grid, letting the environment represent an actual place (initially tested near UC Berkeley, then downtown San Jose) instead of only abstract random grids. Fixed a rasterization bug where a simple touch-test overcounted obstacle density at city scale (95%+ blocked cells), replacing it with a minimum-area-overlap threshold. Fixed a severe performance bug (city-scale queries effectively hung) by adding a spatial index (STRtree) instead of unioning all building geometry up front.

Milestone 3 -- Visualization

Built the matplotlib desktop viewer, added a rotating plane/drone icon (in place of a static dot), added an optional real basemap image underneath the grid via contextily, and added live per-drone battery display.

Milestone 4 -- Decision-making strategies

Replaced random action selection with a hand-coded potential-field policy, diagnosed and fixed multiple oscillation/freezing failure modes, then replaced that with a BFS shortest-path coverage policy once potential fields' structural local-minimum problem proved unfixable by patching alone. Measured and confirmed BFS roughly doubles coverage versus potential fields across 15 test seeds.

Milestone 5 -- Live web interface

Built server.py (FastAPI + WebSocket) to run the simulation continuously and stream live state to a browser, and a Leaflet-based frontend (web/) to render it on a real, interactive map with a live stats sidebar. Verified the server starts cleanly and the WebSocket streams correct data end to end.

Milestone 6 -- Version control

Initialized a git repository, wrote a requirements.txt for reproducibility, excluded the virtual environment and generated map caches from version control, and pushed the project to a remote GitHub repository.

Current Stage Assessment

An honest assessment of what is solid versus what is still missing, as of this report.

Solid and verified

- A correct, PettingZoo-compliant multi-agent environment.
- Real-world obstacle data integration, verified against an actual city area with sane, measured obstacle density.
- A non-learned coverage strategy (BFS) that reliably outperforms both random action selection and a more naive rule-based approach, with measured results, not just claims.
- Two independent, working live visualizations, one of them on a real interactive map.
- Per-drone battery tracking as real, queryable simulation state.

Not yet built -- important to say plainly

- No trained policy exists. Nothing in this project has learned anything from experience yet; BFS is a hand-coded algorithm, not a model.
- No training loop exists. TorchRL is installed but unused -- there is no PPO, no replay buffer, no policy network being updated.
- No real communication model exists (see previous section) -- coordination currently relies on global, instantaneous shared state.
- No obstacle recognition exists -- only ground-truth obstacle lookup within a fixed observation radius (see 'Obstacles' section).

Bottom line

This is currently a well-structured, empirically-tested simulator and visualization stack with a strong non-learned baseline -- not yet a learning or communication-realistic swarm intelligence system. Both of those are the explicit next steps.

Planned Work: Obstacle Recognition on Unknown Terrain

This is the most important planned change to how obstacles work, and it is conceptually distinct from anything implemented so far -- see the 'Obstacles' section for why random generation and real-world placement are both NOT the same thing as recognition.

The problem to solve

Right now, whether obstacles come from random placement (training variety) or real OpenStreetMap data (a specific real place), the drone's observation is built by directly reading a hidden, perfect-knowledge grid -- it is simply told which nearby cells are blocked. There is no step where the system has to figure out, from ambiguous or partial input, whether a given patch of unknown terrain contains an obstacle.

What obstacle recognition would actually mean

- The drone receives something short of ground truth -- e.g. simulated noisy sensor readings, a rendered image patch, or elevation/imagery data -- instead of a clean -1/0/1 lookup.
- A recognition step (classical image processing, a small CNN, or a simpler heuristic depending on scope) has to infer 'is this cell traversable' from that imperfect input.
- Critically, this needs to generalize to terrain the system was not specifically trained on -- 'unknown terrain' -- rather than memorizing one map's obstacle layout.

Why keep this separate from random obstacle generation

Random obstacle generation already exists and already serves the generalization goal for training variety -- a policy trained across many randomly-generated layouts is less likely to overfit to one fixed map. That is a training-data mechanism. Obstacle recognition is a perception mechanism: it is about how a drone figures out where obstacles are in the first place, independent of whether the underlying layout was randomly generated or real. Both random layouts and real OSM layouts could, in principle, be paired with either ground-truth lookup (today) or a real recognition step (planned) -- they are orthogonal design choices, and treating them as one thing would understate the actual scope of this planned work.

Rough approach

Most likely: keep the existing ground-truth grid as the environment's internal state (still needed to compute reward/coverage correctly), but change _get_obs() so it no longer reads that grid directly for obstacle cells -- instead route it through a perception function that receives a noisier or indirect signal and has to infer the obstacle/free classification itself.

Planned Work: Communication and Learning

Realistic drone-to-drone communication constraints

Replace the current global, instantaneous shared state with an explicit message-passing model: each drone maintains its own 'last known' cache of teammate positions/battery, updated only when a simulated broadcast actually reaches it. Planned constraints to model, based on how real swarms operate:

- A limited communication range -- teammates farther than some distance simply don't get an update that step.
- A chance of dropped or delayed messages, rather than perfect delivery.
- Optionally, mesh relaying -- a message can hop through an intermediate drone to reach one outside direct range.

This directly changes what BFS's territory-claiming and potential fields' teammate repulsion are allowed to assume -- both currently read exact, live teammate positions, which would no longer be available under this model.

A trained policy, and a real comparison against BFS

TorchRL (already installed) would be used to train an actual neural-network policy via a standard multi-agent RL loop -- most likely PPO, given the discrete action space and the shared-reward structure already in place. The goal is a direct, measured comparison against the BFS baseline established in this report, on the same maps: does a learned policy discover coordination behavior that outperforms hand-coded pathfinding once communication constraints are added, where BFS's assumption of perfect teammate knowledge no longer holds?

Suggested order of implementation

- 1. Obstacle recognition on unknown terrain (independent of the other two, can be built and tested on its own).
- 2. Communication constraints (changes what any policy -- BFS or learned -- is allowed to know about teammates).
- 3. Trained policy, evaluated under those communication constraints, compared against the BFS baseline under the same constraints.

What Is Innovative About This Project

None of the individual techniques used here are new inventions -- OpenStreetMap obstacle extraction, breadth-first search, potential fields, and PettingZoo environments are all established, well-documented tools and algorithms. The innovative part of this project is in how they are combined and sequenced for this specific problem, not in any single novel algorithm.

Grounding an RL environment in real, verifiable geography

Rather than training/testing against an abstract random grid, obstacles are pulled from real building and water geometry for an actual named place, rasterized correctly (after fixing a real overcounting bug) at city scale. This turns 'our swarm searches a grid' into a claim that can be checked against an actual map -- a stronger and more concrete basis for evaluation than a synthetic benchmark alone.

A rigorous, measured baseline before any learning is attempted

Instead of jumping straight to training a policy and hoping it works, this project built and empirically compared three non-learned strategies (random, potential fields, BFS) on identical test conditions, diagnosed real failure modes in the weaker ones rather than dismissing them, and established BFS as a concrete, reproducible baseline (77.2% mean coverage across 15 seeds) that any future trained policy will need to be measured against and ideally beat. This provides a real performance floor rather than an assumption.

A clean separation that keeps the RL contract usable throughout

Every decision-making strategy built so far -- including the non-learned ones -- goes through the exact same PettingZoo action/observation interface a trained policy will eventually use. None of the visualization, obstacle, or battery work required special-casing around any particular strategy. That discipline is what makes it possible to swap in a trained policy later without reworking the environment, the real-world data pipeline, or either visualization -- which is not guaranteed by default in fast-moving prototype code, and was a deliberate design choice here.

Live, real-map visualization of an RL simulation

Streaming live simulation state from a Python RL environment to a real, interactive web map over WebSocket -- rather than only a static offline plot -- is a genuinely useful and less commonly built piece of infrastructure for this kind of project, and it was built with zero paid API dependencies.